

Part I

1. In HTML, what is the difference between an element and an attribute?
 - In HTML, an element consists of a tag and its content, representing the structure of the webpage. An attribute provides additional information about an element and is defined within the opening tag.
2. Why do we usually separate content (HTML), presentation (CSS), and behavior (JavaScript)?
 - HTML, CSS, and JavaScript each have distinct roles in web development. Separating content, presentation, and behavior allows each component to be managed independently, making the code more organized and easier to maintain.
3. Which lines in your CSS control the page background, heading appearance, and button styling?

```
## For page background
body {
  background: #8fabcc;
}
```

```
## For heading appearance
body {
  font-family: Arial, Helvetica, sans-serif;
  font-size: 16px;
  color: #172033;
}

.hero {
  padding: 2.5rem 1.75rem;
  background: linear-gradient(135deg, #0d1b2a, #2a9d8f);
  color: white;
}
```

```
## For button styling
button {
  background: #bfldd1;
  color: white;
  border: none;
  padding: 0.75rem 1.1rem;
  border-radius: 999px;
  cursor: pointer;
}
button:hover {
  filter: brightness(1.3);
  box-shadow: 0 6px 14px rgba(20, 90, 158, 0.2);
}
```

4. What does the JavaScript event listener do, and what visible change does it trigger on the page?
 - The JavaScript event listener detects when the button is clicked and executes a function. It toggles the visibility of the extra content and updates the button text.

5. Why is it useful to preview your webpage in both the editor and the browser while you work?
 - Previewing the webpage in both the editor and the browser is useful because the editor shows the code structure, while the browser shows how the page actually renders and behaves. This helps identify issues, styling problems, and interaction errors in real time.

Part II

1. Why is `city_id` useful as a shared key across the map, charts, and table?
 - The `'city_id'` serves as a unique identifier that links all components of the dashboard, including the map, charts, and table. It allows interactions to consistently identify the same city across different visualizations.

2. Which function in the notebook is responsible for replacing the placeholder tokens in the HTML template?
 - `render_dashboard()`
 - It reads the HTML template and uses the `replace()` method to substitute each token with the corresponding generated HTML content.

3. Why does this version embed `window.CHART_DATA` before the interaction code runs?
 - Because the JavaScript logic can access the required data immediately. If the data were not defined beforehand, the interaction code would fail because `window.CHART_DATA` would be undefined.

4. What parts of the final dashboard are primarily controlled by Python, which by HTML/CSS, and which by JavaScript?
 - Python controls data processing, chart generation, and template rendering. HTML and CSS control the layout, structure, and visual styling of the dashboard. JavaScript handles user interactions, such as clicks and highlighting.

5. In the interaction code, what happens when a user clicks a point or bar, and how is the selection cleared?
 - When a user clicks a point or bar, the `selectCity` function is called, which highlights the selected city across the map, bar chart, scatter plot, and table. The selection is cleared either by double-clicking a chart or pressing the Escape key, which triggers the `clearSelection` function to reset all visual elements to their original state.